OOSE Batch 1

B.E / B.Tech. PRACTICAL END SEMESTER EXAMINATIONS

Sixth Semester

CCS356 - OBJECT ORIENTED SOFTWARE ENGINEERING LABORATORY

(Regulations 2021) - SET 1 (SIMPLIFIED)

Time: 3 Hours | **Max. Marks:** 100

MARK ALLOCATION TABLE

AIM	PROBLEM STATEMENT	UML DIAGRAMS	IMPLEMENTATION	OUTPUT	VIVA	TOTAL
10	10	30	30	10	10	**100**

General Instructions for UML Diagrams (For all Experiments)

Draw.io Link: Click here to open Draw.io (app.diagrams.net)
How to easily draw: 1. Open Draw.io -> Select "UML" from the left shape panel.
2. **Use Case:** Stick figures for Actors, Ovals for processes (e.g., Login, Register).
3. **Class Diagram:** 3-tier boxes (Top: Class Name, Middle: Attributes, Bottom: Methods).
4. **Sequence Diagram:** Vertical lifelines for objects, horizontal arrows for messages.

Ex 1: Course Reservation System

Aim: To design and implement a Course Reservation System.
Problem Statement: The system must handle registration for courses, selection of courses, and maintain a student database.
UML Diagrams: Open Draw.io

- **Actors:** Student, Admin.
- **Use Cases:** Login, Browse Courses, Select Course, Pay Fees.
- **Classes:** `Student`, `Course`, `Database`.
  Implementation (Java):

```
class Course {
    String courseName;
    public Course(String name) { this.courseName = name; }
}
class Student {
    String name;
    public void registerCourse(Course c) {
        System.out.println("Registered successfully for: " +
c.courseName);
    }
}
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.name = "John";
        s.registerCourse(new Course("OOSE"));
    }
}
```

Output: `Registered successfully for: OOSE`
Result: The Course Reservation System was successfully designed and implemented using UML and Java.

Ex 2: Student Mark Analysis System

Aim: To design and implement a Student Mark Analysis System.
Problem Statement: The system must maintain student details, mark details, and automatically calculate grades and percentages.
UML Diagrams: Open Draw.io

- **Actors:** Teacher, Student, System.
- **Use Cases:** Enter Marks, View Marks, Generate Result.
- **Classes:** `Student`, `MarkSheet`, `GradeCalculator`.
  Implementation (Java):

```
class Student {
    int m1, m2, m3;
    public void calculate() {
        int total = m1 + m2 + m3;
        float percent = total / 3.0f;
        System.out.println("Total: " + total + " | Percentage: " + percent
+ "%");
        if(percent > 80) System.out.println("Grade: A");
        else System.out.println("Grade: B");
    }
```

```
}
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.m1 = 85; s.m2 = 90; s.m3 = 80;
        s.calculate();
    }
}
```

Output: `Total: 255 | Percentage: 85.0% | Grade: A`
Result: The Student Mark Analysis System was successfully designed and implemented.

Ex 3: Stock Maintenance System

Aim: To design and implement a Stock Maintenance System.
Problem Statement: The system must manage inventory control, stock details, and maintain a reorder level.
UML Diagrams: Open Draw.io

- **Actors:** Store Manager, Supplier.
- **Use Cases:** Add Stock, Update Stock, Check Reorder Level, Order New Stock.
- **Classes:** `Item`, `Inventory`, `Supplier`.
  Implementation (Java):

```
class Item {
    String name; int quantity, reorderLevel;
    public void checkStock() {
        if (quantity < reorderLevel) {
            System.out.println("Alert: Stock low for " + name + ". Reorder
immediately.");
        } else {
            System.out.println("Stock normal for " + name);
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Item item = new Item();
        item.name = "Laptops"; item.quantity = 5; item.reorderLevel = 10;
        item.checkStock();
    }
}
```

Output: `Alert: Stock low for Laptops. Reorder immediately.`
Result: The Stock Maintenance System was successfully designed and implemented.

Ex 4: BPO Client System

Aim: To design and implement a BPO Client Management System.
Problem Statement: The system must maintain customer details and track their query details.
UML Diagrams: Open Draw.io

- **Actors:** Customer, BPO Executive.
- **Use Cases:** Log Query, View Customer Info, Resolve Query.
- **Classes:** `Customer`, `QueryTicket`, `Executive`.
  Implementation (Java):

```
class Customer {
    String name, query;
    public void raiseQuery(String q) {
        this.query = q;
        System.out.println(name + " raised query: " + query);
    }
}
class Executive {
    public void resolveQuery(Customer c) {
        System.out.println("Resolved query for " + c.name);
    }
}
public class Main {
    public static void main(String[] args) {
        Customer c = new Customer(); c.name = "Alice";
        c.raiseQuery("Internet not working");
        Executive e = new Executive(); e.resolveQuery(c);
    }
}
```

Output: `Alice raised query: Internet not working` / `Resolved query for Alice`
Result: The BPO Client Management System was successfully designed and implemented.

Ex 5: Software Personnel Management System

Aim: To design and implement a Personnel Management System.
Problem Statement: The system must add, view, and search personnel details according to their ID.
UML Diagrams: Open Draw.io

- **Actors:** HR Manager, Employee.

- **Use Cases:** Add Employee, Search Employee, Update Details.
- **Classes:** `Employee`, `HRSystem`.
  Implementation (Java):

```
import java.util.HashMap;
class HRSystem {
    HashMap<Integer, String> db = new HashMap<>();
    public void addEmp(int id, String name) {
        db.put(id, name); System.out.println("Added: " + name);
    }
    public void search(int id) {
        System.out.println("Employee Found: " + db.get(id));
    }
}
public class Main {
    public static void main(String[] args) {
        HRSystem hr = new HRSystem();
        hr.addEmp(101, "Bob");
        hr.search(101);
    }
}
```

Output: `Added: Bob` / `Employee Found: Bob`
Result: The Software Personnel Management System was successfully designed and implemented.

Ex 6: Book Bank System

Aim: To design and implement a Book Bank System.
Problem Statement: The system must add books (edition, price, author), view books by category, and search for specific books.
UML Diagrams: Open Draw.io

- **Actors:** Librarian, User.
- **Use Cases:** Add Book, Search Book, View by Category.
- **Classes:** `Book`, `LibraryDB`, `User`.
  Implementation (Java):

```
class Book {
    String name, category;
    public Book(String n, String c) { this.name = n; this.category = c; }
}
public class Main {
    public static void main(String[] args) {
```

```
        Book b1 = new Book("Java Basics", "Programming");
        String searchCategory = "Programming";
        if(b1.category.equals(searchCategory)) {
            System.out.println("Found in category: " + b1.name);
        }
    }
}
```

Output: `Found in category: Java Basics`
Result: The Book Bank System was successfully designed and implemented.

Ex 7: Exam Registration System

Aim: To design and implement an Exam Registration Form.
Problem Statement: The system must handle registration for exams based on courses, maintain candidate details, and allow viewing of the exam form.
UML Diagrams: Open Draw.io

- **Actors:** Student, Controller of Exams.
- **Use Cases:** Fill Form, Pay Exam Fee, Generate Hall Ticket.
- **Classes:** `Candidate`, `ExamForm`, `Payment`.
  Implementation (Java):

```
class ExamForm {
    String candidateName, subject;
    public void submitForm() {
        System.out.println("Exam Form submitted for: " + candidateName + "
| Subject: " + subject);
    }
}
public class Main {
    public static void main(String[] args) {
        ExamForm f = new ExamForm();
        f.candidateName = "Charlie"; f.subject = "OOSE";
        f.submitForm();
    }
}
```

Output: `Exam Form submitted for: Charlie | Subject: OOSE`
Result: The Exam Registration System was successfully designed and implemented.

Ex 8: E-Book Management System

Aim: To design and implement an E-Book Management System.
Problem Statement: The system must handle e-book booking details, cancellations, and enquiries.
UML Diagrams: Open Draw.io

- **Actors:** Customer, System Admin.
- **Use Cases:** Search E-Book, Purchase/Book, Cancel Order, Enquiry.
- **Classes:** `Customer`, `EBook`, `Order`.
  Implementation (Java):

```
class Order {
    int orderId; String status;
    public void book() { status = "Booked"; System.out.println("Order " +
orderId + " " + status); }
    public void cancel() { status = "Cancelled"; System.out.println("Order
" + orderId + " " + status); }
}
public class Main {
    public static void main(String[] args) {
        Order o = new Order(); o.orderId = 555;
        o.book();
        o.cancel();
    }
}
```

Output: `Order 555 Booked` / `Order 555 Cancelled`
Result: The E-Book Management System was successfully designed and implemented.

Ex 9: Recruitment System

Aim: To design and implement a Recruitment System.
Problem Statement: The system must register new users, allow them to apply for jobs, and search jobs by company/designation.
UML Diagrams: Open Draw.io

- **Actors:** Applicant, HR Recruiter.
- **Use Cases:** Register, Search Job, Upload Resume, Apply Job.
- **Classes:** `Applicant`, `JobPosting`, `Company`.
  Implementation (Java):

```
class JobPosting {
    String company, role;
    public void displayJob() { System.out.println("Job: " + role + " at "
+ company); }
```

```
}
class Applicant {
    public void apply(JobPosting j) { System.out.println("Successfully
applied for " + j.role); }
}
public class Main {
    public static void main(String[] args) {
        JobPosting j = new JobPosting(); j.company = "Google"; j.role =
"Developer";
        j.displayJob();
        Applicant a = new Applicant(); a.apply(j);
    }
}
```

Output: `Job: Developer at Google` / `Successfully applied for Developer`
Result: The Recruitment System was successfully designed and implemented.

Ex 10 & 18: Credit Card Processing System

Aim: To design and implement a Credit Card Processing System.
Problem Statement: Maintain user details, maintain transactions, and view balance amount.
UML Diagrams: Open Draw.io

- **Actors:** Cardholder, Merchant, Bank Server.
- **Use Cases:** Swipe Card, Verify PIN, Deduct Balance, View Balance.
- **Classes:** `Account`, `Transaction`, `CreditCard`.
  Implementation (Java):

```
class Account {
    double balance = 10000.0;
    public void processTransaction(double amount) {
        if(amount <= balance) {
            balance -= amount;
            System.out.println("Transaction successful. Remaining Balance:
" + balance);
        }
        else {
            System.out.println("Transaction Failed: Insufficient Limit.");
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Account acc = new Account();
```

```
        acc.processTransaction(2500.0);
    }
}
```

Output: `Transaction successful. Remaining Balance: 7500.0`
Result: The Credit Card Processing System was successfully designed and implemented.

Ex 11: Foreign Trading System

Aim: To design and implement a Foreign Trading System.

Problem Statement: The system must track import/export details within the country and maintain trading country details.

UML Diagrams: Open Draw.io

- **Actors:** Trader, Customs Admin.
- **Use Cases:** Add Export, View Imports, Manage Countries.
- **Classes:** `Trader`, `TradeTransaction`, `Country`.

Implementation (Java):

```
class TradeTransaction {
    String type, country, item;

    public void processTrade(String t, String c, String i) {
        this.type = t; this.country = c; this.item = i;
        System.out.println(type + " processed for item: " + item);
        System.out.println("Trading Country: " + country);
    }
}

public class Main {
    public static void main(String[] args) {
        TradeTransaction trade = new TradeTransaction();
        trade.processTrade("Export", "Germany", "Textiles");
    }
}
```

Output:

```
Export processed for item: Textiles
Trading Country: Germany
```

Result: The Foreign Trading System was successfully designed and implemented.

Ex 12: Conference Management System

Aim: To design and implement a Conference Management System.

Problem Statement: The system must manage conference details, college details, and allow viewing of colleges based on the month and year of the conference.

UML Diagrams: Open Draw.io

- **Actors:** Admin, Participant.
- **Use Cases:** Add Conference, Register College, View Schedule.
- **Classes:** `Conference`, `College`, `Schedule`.

Implementation (Java):

```
class Conference {
    String name, month; int year;
    public Conference(String n, String m, int y) { this.name=n;
this.month=m; this.year=y; }
}

public class Main {
    public static void main(String[] args) {
        Conference c = new Conference("Tech Symposium", "May", 2026);
        String searchMonth = "May";

        if(c.month.equals(searchMonth)) {
            System.out.println("Conference Found: " + c.name);
            System.out.println("Scheduled for: " + c.month + " " +
c.year);
        }
    }
}
```

Output:

```
Conference Found: Tech Symposium
Scheduled for: May 2026
```

Result: The Conference Management System was successfully designed and implemented.

Ex 13: Book Management System

Aim: To design and implement a Book Management System.

Problem Statement: The system must maintain user details, book details (edition, price, author), and view books based on specific categories.

UML Diagrams: Open Draw.io

- **Actors:** Librarian, Reader.
- **Use Cases:** Add Book, Search Category, View Details.
- **Classes:** `Book`, `User`, `Library`.

Implementation (Java):

```
class Book {
    String title, author, category; double price;
    public Book(String t, String a, String c, double p) {
        this.title=t; this.author=a; this.category=c; this.price=p;
    }

    public void display() {
        System.out.println("Book: " + title + " by " + author + " | $" +
price);
    }
}

public class Main {
    public static void main(String[] args) {
        Book b = new Book("AI Basics", "Alan Turing", "Technology", 45.0);
        if(b.category.equals("Technology")) {
            System.out.println("Category Match Found:");
            b.display();
        }
    }
}
```

Output:

```
Category Match Found:
Book: AI Basics by Alan Turing | $45.0
```

Result: The Book Management System was successfully designed and implemented.

Ex 14: Passport Automation System

Aim: To design and implement a Passport Automation System.

Problem Statement: The system must handle user details, track passport issuance, and allow users to check the status of their issues.

UML Diagrams: Open Draw.io

- **Actors:** Applicant, Passport Officer.
- **Use Cases:** Apply Passport, Update Status, Check Status.
- **Classes:** `Applicant`, `Passport`, `TrackingSystem`.

Implementation (Java):

```
class Passport {
    String applicantName, status;

    public Passport(String name) { this.applicantName = name; this.status
= "In Progress"; }

    public void checkStatus() {
        System.out.println("Applicant: " + applicantName);
        System.out.println("Passport Status: " + status);
    }
}

public class Main {
    public static void main(String[] args) {
        Passport p = new Passport("Ravi Kumar");
        p.status = "Approved & Dispatched";
        p.checkStatus();
    }
}
```

Output:

```
Applicant: Ravi Kumar
Passport Status: Approved & Dispatched
```

Result: The Passport Automation System was successfully designed and implemented.

Ex 15: Online Course Reservation System

Aim: To design and implement an Online Course Reservation System.

Problem Statement: The system must allow users to browse and reserve online courses efficiently.

UML Diagrams: Open Draw.io

- **Actors:** Student, Instructor.
- **Use Cases:** Browse Courses, Reserve Course, Make Payment.
- **Classes:** `Student`, `Course`, `Reservation`.

Implementation (Java):

```
class Reservation {
    public void reserveCourse(String student, String course) {
        System.out.println("Processing reservation...");
        System.out.println(student + " successfully reserved the course: "
+ course);
    }
}

public class Main {
    public static void main(String[] args) {
        Reservation r = new Reservation();
        r.reserveCourse("Alice", "Python Bootcamp");
    }
}
```

Output:

```
Processing reservation...
Alice successfully reserved the course: Python Bootcamp
```

Result: The Online Course Reservation System was successfully designed and implemented.

Ex 16: Exam Registration System

Aim: To design and implement an Exam Registration System.

Problem Statement: The system must manage student exam course selections, confirm registrations, and generate exam details.

UML Diagrams: Open Draw.io

- **Actors:** Student, Exam Controller.
- **Use Cases:** Register Exam, Pay Fees, Generate Hall Ticket.
- **Classes:** `Student`, `Exam`, `Registration`.

Implementation (Java):

```
class Registration {
    String studentName, subject;

    public void register(String name, String sub) {
        this.studentName = name;
        this.subject = sub;
        System.out.println("Exam Registration Confirmed.");
        System.out.println("Candidate: " + name + " | Subject: " + sub);
    }
}

public class Main {
    public static void main(String[] args) {
        Registration reg = new Registration();
        reg.register("Bob", "Database Management");
    }
}
```

Output:

```
Exam Registration Confirmed.
Candidate: Bob | Subject: Database Management
```

Result: The Exam Registration System was successfully designed and implemented.

Ex 17: SRS for Attendance Maintenance System

Aim: To document the Software Requirement Specification (SRS) for an Attendance Maintenance System.

Problem Statement: To write a complete IEEE standard SRS document covering functional and non-functional requirements for tracking student attendance.

UML Diagrams: Open Draw.io

- **Actors:** Teacher, Student, Admin.
- **Use Cases:** Mark Attendance, Generate Report, View Shortage.

Implementation (SRS Structure):

```
1. Introduction
   1.1 Purpose: Automate student attendance tracking.
   1.2 Scope: Allows teachers to mark attendance and students to view % shortage.
2. Functional Requirements
```

```
   - System must allow staff login.
   - System must calculate monthly attendance percentage.
3. Non-Functional Requirements
   - Security: Passwords must be encrypted.
   - Performance: Update records within 2 seconds.
```

Output:

```
SRS Document Generated Successfully.
```

Result: The Software Requirement Specification for the Attendance Maintenance System was successfully documented.

Ex 18: Credit Card Processing System

Aim: To design and implement a Credit Card Processing System.

Problem Statement: The system must maintain user details, process transactions, and securely view the balance amount.

UML Diagrams: Open Draw.io

- **Actors:** Cardholder, Merchant, Bank.
- **Use Cases:** Swipe Card, Process Payment, View Balance.
- **Classes:** `CreditCard`, `Transaction`, `Account`.

Implementation (Java):

```
class CreditCard {
    double limit = 50000.0;

    public void processTransaction(double amount) {
        if(amount <= limit) {
            limit -= amount;
            System.out.println("Transaction Approved for: $" + amount);
            System.out.println("Remaining Credit Limit: $" + limit);
        } else {
            System.out.println("Transaction Declined: Insufficient
Limit.");
        }
    }
}

public class Main {
    public static void main(String[] args) {
```

```
        CreditCard cc = new CreditCard();
        cc.processTransaction(15000.0);
    }
}
```

Output:

```
Transaction Approved for: $15000.0
Remaining Credit Limit: $35000.0
```

Result: The Credit Card Processing System was successfully designed and implemented.

Ex 19: SRS for Library Management System

Aim: To document the Software Requirement Specification (SRS) for a Library Management System.

Problem Statement: To write a complete IEEE standard SRS document covering the functional and non-functional aspects of book issuance and tracking.

UML Diagrams: Open Draw.io

- **Actors:** Librarian, Member.
- **Use Cases:** Issue Book, Return Book, Calculate Fine.

Implementation (SRS Structure):

```
1. Introduction
   1.1 Purpose: Digitize the cataloging and book issuing process.
   1.2 Scope: Used by university librarians and students.
2. Functional Requirements
   - System must track book availability in real-time.
   - System must calculate overdue fines automatically ($1 per day).
3. Non-Functional Requirements
   - Usability: Simple GUI for quick checkouts.
   - Reliability: Database backup every 24 hours.
```

Output:

```
SRS Document Generated Successfully.
```

Result: The Software Requirement Specification for the Library Management System was successfully documented.

Ex 20: Student Mark Analysis System

Aim: To design and implement a Student Mark Analysis System.

Problem Statement: The system must maintain student details, subject marks, and automatically calculate the percentage and final grade.

UML Diagrams: Open Draw.io

- **Actors:** Teacher, Student, System.
- **Use Cases:** Enter Marks, Calculate Grade, View Results.
- **Classes:** `Student`, `MarkSheet`, `GradeCalculator`.

Implementation (Java):

```
class Student {
    String name; int m1, m2, m3;

    public Student(String n, int a, int b, int c) { this.name=n;
this.m1=a; this.m2=b; this.m3=c; }

    public void generateResult() {
        int total = m1 + m2 + m3;
        float percent = total / 3.0f;
        System.out.println("Student: " + name);
        System.out.println("Percentage: " + percent + "%");

        if(percent >= 90) System.out.println("Grade: O");
        else if(percent >= 80) System.out.println("Grade: A+");
        else System.out.println("Grade: A");
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Charlie", 95, 88, 92);
        s.generateResult();
    }
}
```

Output:

```
Student: Charlie
Percentage: 91.666664%
Grade: O
```

Result: The Student Mark Analysis System was successfully designed and implemented.